

Отчёт по лабораторной работе №3

Компьютерный практикум по статистическому анализу данных

Управляющие структуры

Выполнил: Исаев Булат Абубакарович,
НПИбд-01-22, 1132227131

Содержание

1	Цель работы	1
2	Выполнение лабораторной работы.....	1
2.1	Циклы while и for	1
2.2	Условные выражения	4
2.3	Функции	5
2.4	Сторонние библиотеки (пакеты) в Julia	7
2.5	Самостоятельная работа.....	8
3	Вывод.....	17
4	Список литературы. Библиография	17

1 Цель работы

Основная цель работы — освоить применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

2 Выполнение лабораторной работы

2.1 Циклы while и for

Для различных операций, связанных с перебором индексируемых элементов структур данных, традиционно используются циклы while и for.

Синтаксис while

```
while <условие>  
    <тело цикла>  
end
```

Примеры использования цикла while (рис. 1):

```
•[1]: # пока n<10 прибавить к n единицу и распечатать значение:  
n = 0  
while n < 10  
    n += 1  
    println(n)  
end
```

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

```
•[2]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]  
i = 1  
while i <= length(myfriends)  
    friend = myfriends[i]  
    println("Hi $friend, it's great to see you!")  
    i += 1  
end
```

```
Hi Ted, it's great to see you!  
Hi Robyn, it's great to see you!  
Hi Barney, it's great to see you!  
Hi Lily, it's great to see you!  
Hi Marshall, it's great to see you!
```

Рис. 1: Примеры использования цикла *while*

Такие же результаты можно получить при использовании цикла *for*.

Синтаксис *for*

```
for <переменная> in <диапазон>  
    <тело цикла>  
end
```

Примеры использования цикла *for* (рис. 2):

•[3]: *# Рассмотренные выше примеры, но с использованием цикла for:*

```
for n in 1:2:10
    println(n)
end
```

```
1
3
5
7
9
```

[4]:

```
myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]
for friend in myfriends
    println("Hi $friend, it's great to see you!")
end
```

```
Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!
```

Рис. 2: Примеры использования цикла for

Пример использования цикла for для создания двумерного массива, в котором значение каждой записи является суммой индексов строки и столбца (рис. 3):

```
[14]: # инициализация массива m x n из нулей:
m, n = 5, 5
A = fill(0, (m, n))

# формирование массива, в котором значение каждой записи
# является суммой индексов строки и столбца:
for i in 1:m
    for j in 1:n
        A[i, j] = i + j
    end
end
A
```

```
[14]: 5×5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10
```

```
[15]: # инициализация массива m x n из нулей:
B = fill(0, (m, n))

for i in 1:m, j in 1:n
    B[i, j] = i + j
end
B
```

```
[15]: 5×5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10
```

```
[16]: # Ещё одна реализация этого же примера:

C = [i + j for i in 1:m, j in 1:n]
C
```

```
[16]: 5×5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10
```

Рис. 3: Пример использования цикла `for` для создания двумерного массива

2.2 Условные выражения

Довольно часто при решении задач требуется проверить выполнение тех или иных условий. Для этого используют условные выражения.

Синтаксис условных выражений с ключевым словом:

```
if <условие 1>  
    <действие 1>  
elseif <условие 2>  
    <действие 2>  
else  
    <действие 3>  
end
```

Примеры использования условного выражения (рис. 4):

```
[18]: # используем `&&` для реализации операции "AND"  
      # операция % вычисляет остаток от деления  
  
      N = 50  
  
      if (N % 3 == 0) && (N % 5 == 0)  
          println("FizzBuzz")  
      elseif N % 3 == 0  
          println("Fizz")  
      elseif N % 5 == 0  
          println("Buzz")  
      else  
          println(N)  
      end  
  
      Buzz
```

```
[19]: x = 5  
      y = 10  
      (x > y) ? x : y
```

```
[19]: 10
```

Рис. 4: Примеры использования условного выражения

2.3 Функции

Julia дает нам несколько разных способов написать функцию.

Примеры способов написания функции (рис. 5):

```
[20]: function sayhi(name)
      println("Hi $name, it's great to see you!")
      end
```

функция возведения в квадрат:

```
function f(x)
    x^2
end
```

Вызов функции осуществляется по её имени с указанием аргументов, например:

```
sayhi("C-3P0")
f(42)
```

Hi C-3P0, it's great to see you!

```
[20]: 1764
```

```
[21]: # В качестве альтернативы, можно объявить любую из выше определённых функций в одной строке:
```

```
sayhi2(name) = println("Hi $name, it's great to see you!")
f2(x) = x^2
```

```
sayhi("C-3P0")
f(42)
```

Hi C-3P0, it's great to see you!

```
[21]: 1764
```

```
[22]: # В качестве альтернативы, можно объявить любую из выше определённых функций в одной строке:
```

```
sayhi2(name) = println("Hi $name, it's great to see you!")
f3 = x -> x^2
```

```
sayhi("C-3P0")
f(42)
```

Hi C-3P0, it's great to see you!

```
[22]: 1764
```

Рис. 5: Примеры способов написания функции

По соглашению в Julia функции, сопровождаемые восклицательным знаком, изменяют свое содержимое, а функции без восклицательного знака не делают этого (рис. 6):

```
[25]: # В Julia функция map является функцией высшего порядка, которая принимает функцию
      # в качестве одного из своих входных аргументов и применяет эту функцию к каждому
      # элементу структуры данных, которая ей передаётся также в качестве аргумента.
```

```
f(x) = x^3
map(f, [1, 2, 3])
```

```
[25]: 3-element Vector{Int64}:
```

```
 1
 8
27
```

```
[26]: # Функция broadcast – ещё одна функция высшего порядка в Julia, представляющая собой обобщение функции map. Функция broadcast() будет пытаться привести все объекты
      # к общему измерению, map() будет напрямую применять данную функцию поэлементно.
      # Синтаксис для вызова broadcast такой же, как и для вызовов map, например применение
```

```
f(x) = x^3
broadcast(f, [1, 2, 3])
```

```
[26]: 3-element Vector{Int64}:
```

```
 1
 8
27
```

Рис. 6: Сравнение результатов вывода

В Julia функция map является функцией высшего порядка, которая принимает функцию в качестве одного из своих входных аргументов и применяет эту функцию к каждому элементу структуры данных, которая ей передаётся также в качестве аргумента.

Функция `broadcast` — ещё одна функция высшего порядка в Julia, представляющая собой обобщение функции `map`. Функция `broadcast()` будет пытаться привести все объекты к общему измерению, `map()` будет напрямую применять данную функцию поэлементно.

Примеры использования функций `map()` и `broadcast()` (рис. 7):

```
•[23]: # Например, сравните результат применения sort и sort!:
```

```
# задаём массив v:
v = [3, 5, 2]
sort(v)
v
```

```
[23]: 3-element Vector{Int64}:
      3
      5
      2
```

```
[24]: sort!(v)
      v
```

```
[24]: 3-element Vector{Int64}:
      2
      3
      5
```

Рис. 7: Примеры использования функций `map()` и `broadcast()`

2.4 Сторонние библиотеки (пакеты) в Julia

Julia имеет более 2000 зарегистрированных пакетов, что делает их огромной частью экосистемы Julia. Есть вызовы функций первого класса для других языков, обеспечивающие интерфейсы сторонних функций. Можно вызвать функции из Python или R, например, с помощью `PyCall` или `Rcall`.

С перечнем доступных в Julia пакетов можно ознакомиться на страницах следующих ресурсов: - <https://julialang.org/packages/> - <https://juliahub.com/ui/Home> - <https://juliaobserver.com/> - <https://github.com/svaksha/Julia.jl>

При первом использовании пакета в вашей текущей установке Julia вам необходимо использовать менеджер пакетов, чтобы явно его добавить:

```
import Pkg
Pkg.add("Example")
```

При каждом новом использовании Julia (например, в начале нового сеанса в REPL или открытии блокнота в первый раз) нужно загрузить пакет, используя ключевое слово `using`:

Например, добавим и загрузим пакет `Colors`:

```
Pkg.add("Colors")
using Colors
```

Затем создадим палитру из 100 разных цветов:

```
palette = distinguishable_colors(100)
```

А затем определим матрицу 3×3 с элементами в форме случайного цвета из палитры, используя функцию rand:

```
rand(palette, 3, 3)
```

Пример использования сторонних библиотек (рис. 8):

```
[27]: import Pkg
      Pkg.add("Colors")
      using Colors

      palette = distinguishable_colors(100)

      rand(palette, 3, 3)

      Updating registry at `C:\Users\Пользователь\.julia\registries\General.toml`
      Resolving package versions...
      Installed ColorTypes _____ v0.12.1
      Installed Reexport _____ v1.2.2
      Installed Statistics _____ v1.11.1
      Installed FixedPointNumbers - v0.8.5
      Installed Colors _____ v0.13.1
      Updating `C:\Users\Пользователь\.julia\environments\v1.12\Project.toml`
      [5ae59095] + Colors v0.13.1
      Updating `C:\Users\Пользователь\.julia\environments\v1.12\Manifest.toml`
      [3da002f7] + ColorTypes v0.12.1
      [5ae59095] + Colors v0.13.1
      [53c48c17] + FixedPointNumbers v0.8.5
      [189a3867] + Reexport v1.2.2
      [10745b16] + Statistics v1.11.1
      [37e2e46d] + LinearAlgebra v1.12.0
      [e66e0078] + CompilerSupportLibraries_jll v1.3.0+1
      [4536629a] + OpenBLAS_jll v0.3.29+0
      [8e850b90] + libblastrampoline_jll v5.13.1+1
      Precompiling packages...
      750.3 ms ✓ Statistics
      987.5 ms ✓ Reexport
      1951.5 ms ✓ FixedPointNumbers
      1523.1 ms ✓ ColorTypes
      639.5 ms ✓ ColorTypes → StyledStringsExt
      4161.5 ms ✓ Colors
      6 dependencies successfully precompiled in 10 seconds. 40 already precompiled.
```



Рис. 8: Пример использования сторонних библиотек

2.5 Самостоятельная работа

Выполнение задания №1 (рис. 9 - рис. 12):


```

•[28]: # while
n = 1
while n <= 100
    println("$n^2 = $(n^2)")
    n += 1
end

```

1^2 = 1
 2^2 = 4
 3^2 = 9
 4^2 = 16
 5^2 = 25
 6^2 = 36
 7^2 = 49
 8^2 = 64
 9^2 = 81
 10^2 = 100
 11^2 = 121
 12^2 = 144
 13^2 = 169
 14^2 = 196
 15^2 = 225

Рис. 9: Выполнение подпунктов задания №1

```

[13]: # for

for n in 1:100
    println("$n^2 = $(n^2)")
    n += 1
end

```

1^2 = 1
 2^2 = 4
 3^2 = 9
 4^2 = 16
 5^2 = 25
 6^2 = 36
 7^2 = 49
 8^2 = 64
 9^2 = 81
 10^2 = 100

Рис. 10: Выполнение подпунктов задания №1

```

[30]: squares = Dict{<
for n in 1:100
    squares[n] = n^2
end
println(squares)

```

Dict{Any, Any}{5 => 25, 56 => 3136, 35 => 1225, 55 => 3025, 60 => 3600, 30 => 900, 32 => 1024, 6 => 36, 67 => 4489, 45 => 2025, 73 => 5329, 64 => 4096, 90 => 8100, 4 => 16, 13 => 169, 54 => 2916, 63 => 3969, 86 => 7396, 91 => 8281, 62 => 3844, 58 => 3364, 52 => 2704, 12 => 144, 28 => 784, 75 => 5625, 23 => 529, 92 => 8464, 41 => 1681, 43 => 1849, 11 => 121, 36 => 1296, 68 => 4624, 69 => 4761, 98 => 9604, 82 => 6724, 85 => 7225, 39 => 1521, 84 => 7056, 77 => 5929, 7 => 49, 25 => 625, 95 => 9025, 71 => 5041, 66 => 4356, 76 => 5776, 34 => 1156, 50 => 2500, 59 => 3481, 93 => 8649, 2 => 4, 10 => 100, 18 => 324, 26 => 676, 27 => 729, 42 => 1764, 87 => 7569, 100 => 10000, 79 => 6241, 16 => 256, 20 => 400, 81 => 6561, 19 => 361, 49 => 2401, 44 => 1936, 9 => 81, 31 => 961, 74 => 5476, 61 => 3721, 29 => 841, 94 => 8836, 46 => 2116, 57 => 3249, 70 => 4900, 21 => 441, 38 => 1444, 88 => 7744, 78 => 6084, 72 => 5184, 24 => 576, 8 => 64, 17 => 289, 37 => 1369, 1 => 1, 53 => 2809, 22 => 484, 47 => 2209, 83 => 6889, 99 => 9801, 89 => 7921, 14 => 196, 3 => 9, 80 => 6400, 96 => 9216, 51 => 2601, 33 => 1089, 40 => 1600, 48 => 2304, 15 => 225, 65 => 4225, 97 => 9409}

Рис. 11: Выполнение подпунктов задания №1

```
[31]: squaresArr = [n^2 for n in 1:100]
println(squaresArr)

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324,
361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 961, 1024, 1089, 1
156, 1225, 1296, 1369, 1444, 1521, 1600, 1681, 1764, 1849, 1936, 2025, 2116, 2
209, 2304, 2401, 2500, 2601, 2704, 2809, 2916, 3025, 3136, 3249, 3364, 3481, 3
600, 3721, 3844, 3969, 4096, 4225, 4356, 4489, 4624, 4761, 4900, 5041, 5184, 5
329, 5476, 5625, 5776, 5929, 6084, 6241, 6400, 6561, 6724, 6889, 7056, 7225, 7
396, 7569, 7744, 7921, 8100, 8281, 8464, 8649, 8836, 9025, 9216, 9409, 9604, 9
801, 10000]
```

Рис. 12: Выполнение подпунктов задания №1

Выполнение задания №2 (рис. 13):

```
[32]: # Условный оператор
n = 42
if n % 2 == 0
    println(n)
else
    println("Нечёт")
end

# Условный оператор
println(n % 2 == 0 ? n : "Нечёт")

42
42
```

Рис. 13: Выполнение задания №2

Выполнение задания №3 (рис. 13):

```
[34]: function addOne(x)
        return x + 1
    end
println(addOne(3))

4
```

Рис. 14: Выполнение задания №3

Выполнение задания №4 (рис. 15):

```
[35]: A = reshape(1:9, 3, 3)
      B = map(x -> x + 1, A)
      println(B)
```

```
[2 5 8; 3 6 9; 4 7 10]
```

```
[36]: B = broadcast(x -> x+1, A)
      println(B)
```

```
[2 5 8; 3 6 9; 4 7 10]
```

Рис. 15: Выполнение задания №4

Выполнение задания №5 (рис. 16):

```
[40]: A = [1 1 3; 5 2 6; -2 -1 -3]
```

```
println(map(x -> x^3, A))
```

```
[1 1 27; 125 8 216; -8 -1 -27]
```

```
[41]: A[:, 3] = A[:, 2] + A[:, 3]
      println(A)
```

```
[1 1 4; 5 2 8; -2 -1 -4]
```

Рис. 16: Выполнение задания №5

Выполнение задания №6 (рис. 17):

```
[42]: B = repeat([10 -10 10], 15, 1)
```

```
[42]: 15×3 Matrix{Int64}:
```

```
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
 10  -10  10
```

Рис. 17: Выполнение задания №6

Выполнение задания №7 (рис. 18 - рис. 19):

```
[44]: function printMatrix(matrix)
    for row in eachrow(matrix)
        println(row)
    end
    println()
end

Z = zeros{Int, 6, 6}
println("Матрица Z: ")
printMatrix(Z)

Z1 = copy(Z)
for i in 1:6
    for j in 1:6
        if abs(i - j) == 1
            Z1[i, j] = 1
        end
    end
end

Z4 = copy(Z)
for i in 1:6
    for j in 1:6
        if abs(i - j) == 1
            Z4[i, j] = 1
        end
    end
end

println("Матрица Z1: ")
printMatrix(Z1)
println("Матрица Z4: ")
printMatrix(Z4)
```

Рис. 18: Выполнение задания №7

Матрица Z:

```
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0]
```

Матрица Z1:

```
[0, 1, 0, 0, 0, 0]
[1, 0, 1, 0, 0, 0]
[0, 1, 0, 1, 0, 0]
[0, 0, 1, 0, 1, 0]
[0, 0, 0, 1, 0, 1]
[0, 0, 0, 0, 1, 0]
```

Матрица Z4:

```
[0, 1, 0, 0, 0, 0]
[1, 0, 1, 0, 0, 0]
[0, 1, 0, 1, 0, 0]
[0, 0, 1, 0, 1, 0]
[0, 0, 0, 1, 0, 1]
[0, 0, 0, 0, 1, 0]
```

Рис. 19: Выполнение задания №7

Выполнение задания №8 (рис. 20 - рис. 22):

```
[48]: function outer(x, y, operation)
      return [operation(xi, yj) for xi in x, yj in y]
      end
```

```
[48]: outer (generic function with 1 method)
```

Рис. 20: Выполнение подпунктов задания №8

```
[49]: # Матрица сложения элементов
A1 = outer(0:4, 0:4, +)

# Функция возведения в степень ( $0^0 = 0$ )
function safePow(x, y)
    x == 0 && y == 0 ? 0 : x^y
end

# Матрица с пропуском первого элемента
A2 = [j == 1 ? i : safePow(i, j) for i in 0:4, j in 1:5]

# Матрица с циклическим сдвигом по модулю 5
A3 = outer(0:4, 0:4, (x, y) -> mod(x + y, 5))

# Матрица с циклическим сдвигом по модулю 10
A4 = outer(0:9, 0:9, (x, y) -> mod(x + y, 10))

# Матрица с разницей по модулю 9
A5 = outer(0:8, 0:8, (x, y) -> mod(x - y, 9))

# Функция печати матрицы
function printMatrix(name, matrix)
    println("\n")
    println("Матрица $name: ")
    for row in eachrow(matrix)
        println(row)
    end
end

printMatrix("A1", A1)
printMatrix("A2", A2)
printMatrix("A3", A3)
printMatrix("A4", A4)
printMatrix("A5", A5)
```

Рис. 21: Выполнение подпунктов задания №8

Матрица A1:

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[2, 3, 4, 5, 6]
[3, 4, 5, 6, 7]
[4, 5, 6, 7, 8]
```

Матрица A2:

```
[0, 0, 0, 0, 0]
[1, 1, 1, 1, 1]
[2, 4, 8, 16, 32]
[3, 9, 27, 81, 243]
[4, 16, 64, 256, 1024]
```

Матрица A3:

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 0]
[2, 3, 4, 0, 1]
[3, 4, 0, 1, 2]
[4, 0, 1, 2, 3]
```

Матрица A4:

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[1, 2, 3, 4, 5, 6, 7, 8, 9, 0]
[2, 3, 4, 5, 6, 7, 8, 9, 0, 1]
[3, 4, 5, 6, 7, 8, 9, 0, 1, 2]
[4, 5, 6, 7, 8, 9, 0, 1, 2, 3]
[5, 6, 7, 8, 9, 0, 1, 2, 3, 4]
[6, 7, 8, 9, 0, 1, 2, 3, 4, 5]
[7, 8, 9, 0, 1, 2, 3, 4, 5, 6]
[8, 9, 0, 1, 2, 3, 4, 5, 6, 7]
[9, 0, 1, 2, 3, 4, 5, 6, 7, 8]
```

Матрица A5:

```
[0, 8, 7, 6, 5, 4, 3, 2, 1]
[1, 0, 8, 7, 6, 5, 4, 3, 2]
[2, 1, 0, 8, 7, 6, 5, 4, 3]
[3, 2, 1, 0, 8, 7, 6, 5, 4]
[4, 3, 2, 1, 0, 8, 7, 6, 5]
[5, 4, 3, 2, 1, 0, 8, 7, 6]
[6, 5, 4, 3, 2, 1, 0, 8, 7]
[7, 6, 5, 4, 3, 2, 1, 0, 8]
[8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Рис. 22: Выполнение подпунктов задания №8

Выполнение задания №9 (рис. 23):

```
[50]: A = [1 2 3 4 5;
          2 1 2 3 4;
          3 2 1 2 3;
          4 3 2 1 2;
          5 4 3 2 1]
b = [7, -1, -3, 5, -6]

x = A \ b
println(x)

[-3.916666666666667, 3.0000000000000013, 5.0, -9.500000000000002, 5.583333333333335]
```

Рис. 23: Выполнение задания №9

Выполнение задания №10 (рис. 24 - рис. 25):

```
[51]: function printMatrix(name, matrix)
        println("\n")
        println("Матрица $name: ")
        for row in eachrow(matrix)
            println(row)
        end
    end

M = rand(1:10, 6, 10)
printMatrix(M)

[7, 10, 7, 5, 9, 9, 4, 6, 10, 8]
[4, 5, 10, 10, 4, 5, 9, 4, 4, 10]
[2, 6, 6, 1, 8, 2, 8, 8, 4, 1]
[7, 10, 10, 3, 4, 9, 4, 1, 9, 1]
[5, 3, 7, 9, 1, 7, 4, 9, 4, 1]
[6, 3, 1, 4, 3, 9, 8, 2, 3, 6]
```

Рис. 24: Выполнение подпунктов задания №10

```
[55]: K = 75
col_pairs = []
for i in 1:size(M, 2)-1
    for j in i+1:size(M, 2)
        if sum(M[:, i] .+ M[:, j]) > K
            push!(col_pairs, (i, j))
        end
    end
end

println(col_pairs)

Any[(2, 3), (6, 7)]
```

Рис. 25: Выполнение подпунктов задания №10

Выполнение задания №11 (рис. 26):


```
[56]: sum1 = sum(i^4 * (3 + j) for i in 1:20 for j in 1:5)
      println(sum1)
      21679980

[58]: sum2 = sum(i^4 * (3 + i * j) for i in 1:20 for j in 1:5)
      println(sum2)
      195839490
```

Рис. 26: Выполнение задания №11

3 Вывод

В ходе выполнения лабораторной работы было освоено применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

4 Список литературы. Библиография

[1] Julia Documentation: <https://docs.julialang.org/en/v1/>